

Assignment 1

1. Assignment Description

Problem1 : Magic Square Construction (50 points)

A magic square is a square matrix in which the sum of every row, every column, and both main diagonals are the same. Your task is to construct an $N \times N$ magic square for odd values of N .

The numbers from 1 to N^2 are arranged so that:

- The sum of every row is the same.
- The sum of every column is the same.
- The sum of both diagonals is the same.

For example, in the 3×3 magic square:

every row	every column	both diagonal
$6 + 1 + 8 = 15$	$6 + 7 + 2 = 15$	$8 + 5 + 2 = 15$
$7 + 5 + 3 = 15$	$1 + 5 + 9 = 15$	$6 + 5 + 4 = 15$
$2 + 9 + 4 = 15$	$8 + 3 + 4 = 15$	

A simple algorithm for constructing an odd-order magic square is described below.

1. Place 1 in the middle of the first row. Set $k=2$.
2. If the number $k-1$ is placed in the i^{th} row and the j^{th} column, written as position (i, j) , then the number k should be placed in position $(i-1, j-1)$. Note that this matrix is circular.
3. If a number has been previously placed in position $(i-1, j-1)$, then place k in position $(i+1, j)$ instead.
4. Set $k=k+1$
5. Repeat step 2 through 4 until all N^2 numbers are placed in the matrix.

Note that the constructed magic square is symmetric and can be rotated by 90° , 180° , or 270° while still maintaining the magic property.

Input

- The first line contains an integer T , representing the number of test cases.
- Each test case contains one integer N , representing the size of the magic square.
- Formatting Rules
 - **No trailing commas:** Do not place a comma at the end of any line.
 - **No spaces:** The output must not contain any space characters (e.g., print $8, 1, 6$ without spaces).
 - **Newline character:** Use $\backslash n$ (LF) for line breaks. Avoid using $\backslash r \backslash n$ (CRLF).
 - **No newline at EOF:** Do not print a newline character at the very end of your final output.

Output

- For each test case, print the constructed $N \times N$ magic square.
- Each row should be printed on a new line.
- Numbers in the same row must be separated by a comma.

Sample Input

```
2
3
5
```

Sample Output

```
6, 1, 8
7, 5, 3
2, 9, 4
15, 8, 1, 24, 17
16, 14, 7, 5, 23
22, 20, 13, 6, 4
3, 21, 19, 12, 10
9, 2, 25, 18, 11
```

Problem 2: Factorial Numbers (50 points)

Computing factorial numbers is a common problem in mathematics and computer science. The factorial of a non-negative integer N is defined as:

$$N! = N \times (N - 1) \times (N - 2) \times \dots \times 1$$

However, factorial values grow very quickly. Standard integer data types (such as 32-bit or 64-bit integers) cannot store very large factorial values.

For example:

- $12! = 479001600$ (fits in a 32-bit integer)
- $13! = 6227020800$ (exceeds a 32-bit integer)

Since factorial numbers become extremely large, the program should simulate large integer multiplication using an array where:

- Each element stores one digit (0–9)
- The least significant digit is stored at index 0
- Carry operations must be handled during multiplication

For example, the number 3264 can be stored as:

- $A[3] = 3, A[2] = 2, A[1] = 6, A[0] = 4$

The array represents the number 3264, where the digits are stored in reverse order.

Input

- The first line contains an integer T , the number of test cases.
- Each test case contains one integer N ($1 \leq N \leq 50$).

Output

- For each test case, output the factorial value of **N**.
- The factorial must be printed **exactly without overflow**, meaning every digit must be correct.

Sample Input

```
2
15
17
```

Sample Output

```
1307674368000
355687428096000
```

2. Submission Guidelines

Naming and Packaging Conventions

Please organize your code and rename your files according to the following format before uploading. You are required to submit exactly 4 files in total.

- Source Code Archive (.zip): Place your `main` function code, all custom .cpp and .h files, and any other files required for compilation into a specific folder (named according to the Naming Conventions below). Compress this folder into a .zip file. (Note: If you organize your source codes into multiple subdirectories, you must include a `Makefile` in the root of your folder.)
- Compiled Executable File (.exe): Submit the compiled executable file alongside your .zip archive.
- Naming Conventions:
 - For Problem 1 (Magic Square Construction):
 - Folder Name: 1142DS_<Student_ID>_a1a
 - Zip File: 1142DS_<Student_ID>_a1a.zip (e.g., 1142DS_B12345678_a1a.zip)
 - Executable: 1142DS_<Student_ID>_a1a.exe (e.g., 1142DS_B12345678_a1a.exe)
 - For Problem 2 (Factorial Numbers):
 - Folder Name: 1142DS_<Student_ID>_a1b
 - Zip File: 1142DS_<Student_ID>_a1b.zip (e.g., 1142DS_B12345678_a1b.zip)
 - Executable: 1142DS_<Student_ID>_a1b.exe (e.g., 1142DS_B12345678_a1b.exe)
- Total Required Uploads (4 files):

1. 1142DS_<Student_ID>_a1a.zip
2. 1142DS_<Student_ID>_a1a.exe
3. 1142DS_<Student_ID>_a1b.zip
4. 1142DS_<Student_ID>_a1b.exe

Source Code Header Requirement

Every source file (.cpp, .h) you create **must** include a standard header block at the very top of the file. This header provides essential metadata about your code.

Your file header must contain at least the following information:

- **Author:** Your full name and Student ID.
- **Date:** The date the code was written or last modified (e.g., Sept. 22, 2025).
- **Purpose:** A brief description of the program's function and purpose.

File Header Example:

```
/* *****  
 * Author: 名字(B11234567)  
 * Date: Sept. 22, 2025  
 * Purpose: Constructs an N x N magic square for odd N  
 * ***** */
```

3. Grading Policies

General Criteria (Applies to both problems, 15 points, per problem):

- Coding Style (10 points): Assessed based on code readability, consistent indentation, and meaningful naming conventions. Arbitrary or meaningless variable names (e.g., a, b, c, aa, bb, cc) are strictly prohibited and will result in point deductions.
- Comments (5 points): Assessed based on brief and clear explanations of your function purposes and the overall processing flow. Focus on explaining the core logic. Do not add trivial or redundant comments for basic C++ syntax (e.g., commenting on `#include` or `using namespace std`).

Problem-Specific Criteria (35 points, per problem):

- Problem 1: Magic Square Construction (35 points)
 - Correctness (30 points): Points are awarded proportionally based on the percentage of hidden and public test cases your program successfully passes.
 - Class Encapsulation & Dynamic Allocation (5 points): You must encapsulate the Magic Square implementation within a C++ Class. The class must include a constructor, a destructor, and appropriate member functions. Furthermore, the internal array used to store the magic square must be dynamically allocated based on the input size N.

- Problem 2: Factorial Numbers (35 points)
 - Correctness (30 points): Points are awarded proportionally based on the percentage of test cases passed, categorized into the following three difficulty tiers (10 points each):
 - 10 points: Correctly computes $N!$ for $N < 21$ (Results fit within a standard 64-bit unsigned integer limit).
 - 10 points: Correctly computes $N!$ for $21 \leq N \leq 35$ (Results exceed the capacity of a single 64-bit unsigned integer).
 - 10 points: Correctly computes $N!$ for $N > 35$ (Results exceed the capacity of two 64-bit unsigned integers, strictly requiring big integer array manipulation).
 - Class Encapsulation & Operator Overloading (5 points): You must encapsulate the big integer data and operations into a C++ Class. To compute the factorial, you are required to overload operators (e.g., `operator*` and `operator=`) to handle the multiplication of large numbers.

4. Compilation Environment and Restrictions

Compilation Environment

- All source code must comply with the C++17 standard.
- The grading will compile your code using `g++` (e.g., `g++ -std=c++17 ...`). Ensure your code compiles successfully under this environment before submission.

Restrictions (Not Allowed)

Unless explicitly permitted by the assignment prompt, the usage of container libraries in the C++ Standard Template Library (STL)—such as `vector`, `list`, `map`, `set`, `deque`, or similar container structures—is strictly prohibited. If such libraries are used, the correctness score will be automatically **marked as 0**.